Painel do Negócio — Manual técnico e funcional

Documento de passagem do projeto para manutenção, evolução ou reconstrução.

Atualizado: 06/11/2026 (Africa/Maputo).

Código de referência: `53fa8d7`, branch de publicação `main`. Esta edição consolida o manual, preparação comercial, relatórios de intervenção, evolução UI/UX, província e taxa de entrega.

Repositório: https://github.com/BRAVOS23/Painel-Shopify-

Aplicação publicada: https://painel-shopify.lobojose348.workers.dev/

Este documento descreve a implementação existente, as decisões confirmadas pelo proprietário e as limitações conhecidas. Não contém tokens, palavras-passe, chaves privadas, backups de clientes ou valores de Secrets. Configuração instalada numa plataforma não deve ser presumida apenas porque aparece num exemplo do repositório: é necessário verificá-la com o proprietário e na consola correspondente.

Referência principal: este manual substitui afirmações de estado antigas de README.md, SETUP.md e CLOUDFLARE.md, quando houver divergência. docs/UI-UX.md conserva também o histórico das intervenções; entradas antigas desse histórico não representam necessariamente a interface atual.

Documento único: todos os procedimentos necessários estão reunidos nesta edição. Os documentos complementares antigos mantêm valor histórico; esta versão prevalece nas descrições de funcionamento atual. Não é necessário consultá-los para compreender as funcionalidades aqui descritas.

Índice

1. Objetivo e decisões de produto . . . 5

2. Funcionalidades e experiência de utilização . . . 5

2.1 Acesso e conta . . . 5

2.2 Resumo . . . 6

2.3 Pedidos: Lista, Entregas e Relatórios . . . 6

2.4 Produtos . . . 6

2.5 Resultados e Gastos . . . 7

2.6 Província do cliente . . . 7

2.7 Mobile, acessibilidade e instalação . . . 7

3. Regras de pedidos, vendas e stock . . . 7

3.1 Estados e confirmação de venda . . . 7

3.2 Concluir e Cancelar . . . 8

3.3 Reagendamento e datas . . . 8

3.4 Catálogo e stock . . . 9

4. Cálculos financeiros e gráficos . . . 9

5. Arquitetura e mapa dos ficheiros . . . 10

6. Modelo de dados e concorrência . . . 13

6.1 Organização Firestore . . . 13

6.2 Campos de negócio . . . 14

6.3 Ligações e jobs . . . 14

6.4 Gravação e conflitos . . . 15

7. API e contratos . . . 15

8. Integração Shopify . . . 17

8.1 Instalação e OAuth . . . 17

8.2 Importação e webhooks . . . 17
8.3 Importação histórica explícita . . . 18
8.4 Verificação de contagens antigas . . . 18
8.5 Outra loja e troca do aplicativo . . . 19
9. Integração Meta Ads . . . 19
10. Segurança, autenticação e privacidade . . . 20
11. Configuração e ambientes . . . 21
11.1 Identificadores e variáveis públicas . . . 21
11.2 Secrets . . . 21
11.3 Administração e ativação comercial . . . 22
12. Desenvolvimento, compilação e publicação . . . 22
12.1 Arranque local . . . 22
12.2 Build . . . 23
12.3 Publicação . . . 23
13. Testes e critérios de aceitação . . . 23
14. Operação e diagnóstico . . . 25
14.1 Agendamento e quotas . . . 25
14.2 Falhas comuns . . . 26
15. Backups, migrações e recuperação . . . 27
16. Limitações e trabalho pendente . . . 27
17. Como reconstruir do zero . . . 28
Etapa 1 — Especificação e dados . . . 28
Etapa 2 — Identidade e isolamento . . . 28
Etapa 3 — Domínio e funções puras . . . 28
Etapa 4 — Interface móvel primeiro . . . 29
Etapa 5 — Shopify . . . 29

Etapa 6 — Meta . . . 29
Etapa 7 — Catálogo e inventário local . . . 29
Etapa 8 — Migração e comparação . . . 29
Etapa 9 — Operação e publicação . . . 29
Etapa 10 — Continuidade . . . 29
18. Lista de passagem ao próximo programador . . . 29
19. Taxa de entrega padrão e exceções . . . 30
Histórico e exceções . . . 30
Implementação . . . 31
Backup e testes . . . 31
20. Preparação comercial, administração e operação . . . 31
20.1 Estado e decisões . . . 31
20.2 Administração e recebimentos manuais . . . 32
20.3 Sequência de abertura ao público . . . 32
20.4 Pendências externas e capacidade . . . 32
20.5 Exportação completa e restauro operacional . . . 33
21. Histórico consolidado e estado de confirmação . . . 34
22. Manutenção deste documento e ficheiro para download . . . 34

1. Objetivo e decisões de produto

O painel gere um negócio de comércio eletrónico com pagamento na entrega, também chamado COD. Centraliza pedidos e organização de entregas, catálogo e stock local, custos, gastos com publicidade e estimativas de rentabilidade. A interface está em português; a moeda do negócio é MZN, apresentada como MT, e o fuso definido para o negócio é Africa/Maputo.

A Shopify é a origem dos pedidos e dos nomes de produtos vinculados. A Meta fornece gastos com anúncios. O painel acrescenta organização local, reagendamentos, custos dos produtos, entregadores e stock por local.

Decisões confirmadas que devem ser preservadas:

- Orçamento atual zero: Cloudflare Workers Free e Firebase Spark. Não ativar planos pagos automaticamente para resolver falhas.
- Navegação principal: Resumo, Pedidos, Produtos e Resultados. Conta, integrações, aparência e dados ficam no menu de utilizador ⋯.
- A página de fluxo de caixa deixou de ser a entrada do produto. Os movimentos antigos continuam armazenados e incluídos nos backups.
- Reagendamentos podem atravessar meses sem perder o pedido nem o percurso anterior.
- Nomes de produtos podem acompanhar a Shopify; custo, fotografia e stock local pertencem ao painel. O inventário Shopify não deve ser sincronizado para o stock local.
- Uma compra/conversão reportada pela Meta não é uma venda do painel.
- Concluir um pedido ligado à Shopify deve solicitar processamento e registo de pagamento recebido, e aguardar confirmação dos dois estados.
- Os diálogos nativos recorrentes de confirmação de Concluir/Cancelar Shopify foram retirados. Não reintroduzir estes avisos por rotina. Há confirmações em outras ações, como apagar, importar e desligar.
- A limpeza em massa de registos manuais foi retirada. A restauração de backups produzidos por essa antiga funcionalidade permanece disponível.
- Resultados reúne análise financeira e Gastos. O botão Registar gasto abre um diálogo; não existe uma subaba de registo separada. Pedidos separa Lista, Entregas e Relatórios para evitar sobreposição de tarefas.

O painel é um piloto publicado e utilizado pelo proprietário. Não é uma plataforma comercial completa com assinaturas, faturação, equipas e requisitos de distribuição integralmente resolvidos.

2. Funcionalidades e experiência de utilização

2.1 Acesso e conta

Login por email/palavra-passe, criação de conta, recuperação de acesso e verificação de email. A API de produção exige email verificado. Cada UID Firebase representa um negócio, com uma ligação Shopify e uma conta de anúncios Meta. Não existe gestão de várias lojas num login, membros convidados ou equipas; uma segunda loja deve usar outra conta para separar dados.

Uma sessão reconhecida com dados indisponíveis mostra recuperação com Tentar novamente e Entrar noutra conta, sem confundir falhas de login com integração. O menu ⋯ contém Conta, Integrações e Dados: aparência, instalação PWA, plano/acesso, taxa padrão de entrega, termos/privacidade,

exportação, encerramento e migrações. A administração aparece apenas para UIDs configurados no servidor. Sair é impedido enquanto há alterações por guardar.

2.2 Resumo

Pedidos em aberto, entregas concluídas, receita e lucro estimado para o mês escolhido, próximas entregas e atalhos para criar pedidos, reagendar, consultar produtos e gerir integrações. Alerta para remarcados de meses anteriores, verificações pendentes e custos de produto em falta.

2.3 Pedidos: Lista, Entregas e Relatórios

Lista é a consulta por período, estado e pesquisa, incluindo meses anteriores. No móvel, cada cartão recolhido mostra data, estado, nome/telefone, produto, valor e província quando existente. A seta abre contactos rápidos, endereço, origem, quantidade, entregador e ações. Concluir, Cancelar, Reagendar, Editar e Apagar permanecem disponíveis conforme o estado/permissão. A expansão é conservada ao redesenhar a lista. O computador mantém a apresentação tabular. O CSV segue os filtros da lista.

Entregas concentra a organização por mês e grupos Semana 1 a Semana 4, Remarketing e Províncias, registo/edição e pedidos da semana. Novo pedido e Editar abrem esta vista e o formulário certo. Contacto, local, província, data, produto e valor são campos principais; origem, nota, entregador, custo de entrega e estado ficam em Mais detalhes. O custo permite padrão, valor específico ou conservação dos valores legados. O período de consulta da Lista não aparece nesta vista, que usa o seu mês/grupo próprio.

A seleção de vários pedidos para PDF pertence à semana/grupo em consulta e é limpa ao mudar de semana. Reagendamento individual ou em lote pode atravessar meses, conservando a data original e o percurso. Os atalhos de Resumo para uma semana levam à organização de Entregas.

Relatórios contém indicadores de pedidos, conclusões, cancelamentos, remarcados, taxa de sucesso, gráficos de evolução/estado/origem, unidades e entregas por pessoa. A receita e os valores financeiros não são apresentados como destaque nesta vista; a análise financeira fica em Resultados. Verificar contagem na Shopify revê estados já guardados. Importar pedidos criados no período carrega histórico em falta por lotes; são operações diferentes.

O seletor de período é partilhado entre Lista e Relatórios; cada um mostra a sua tarefa em separado. O intervalo inicial é este mês. A antiga aba Meses anteriores e o respetivo código de leitura permanecem como legado, sem entrada na navegação. A nova consulta usa o período da lista. Não reintroduzir essa aba ou caixas semanais dentro da Lista sem uma revisão de produto.

2.4 Produtos

Área principal própria com catálogo pesquisável, cartões de nome/foto/stock e detalhes expansíveis. Adicionar/editar usa diálogo. Custo unitário, fotografia comprimida e stock por local são próprios do painel. Vincular produtos Shopify associa IDs/títulos e preserva estes valores. Aliases associam nomes antigos sem reescrever os registos históricos.

No móvel, a grelha e cada cartão têm limites de largura, nomes quebram por palavras e a pesquisa ocupa a largura disponível. Produtos/unidades e valor em mercadoria têm resumo compacto; o valor em stock usa uma linha própria. A quantidade exibida é o stock local introduzido, não o inventário Shopify.

2.5 Resultados e Gastos

Resultados apresenta período, filtro por produto cadastrado, receita, lucro estimado, ROI, CPA, ROAS, custos em falta, guia de leitura e custos discriminados recolhíveis. Receita e gasto total usam linhas interativas e modo Acumulado; alternar esse modo redesenha a série local sem depender de nova consulta. A distribuição de custos usa gráfico circular com total. Custo de entrega tem um atalho para configurar a taxa padrão na Conta.

Gastos reúne valores manuais e Meta, pesquisa, categoria e total. Registar gasto abre diálogo com uma categoria de cada vez: Anúncios, Outros ou Loja. Anúncios/Outros admitem intervalo inclusivo; Loja admite único ou mensal recorrente. Não existe uma subaba Registar gastos separada nem Produtos dentro de Resultados.

Gastos Meta identificam a campanha e são geridos pela sincronização; não há edição/apagamento manual do seu valor, que seria reposto na próxima importação. Gastos manuais continuam editáveis e removíveis individualmente. O sistema não apaga automaticamente gastos manuais que possam duplicar a Meta.

2.6 Província do cliente

É um campo opcional separado de local. A API pede shippingAddress.province e customAttributes; usa o endereço prioritariamente e atributos com chave de província como alternativa. Uma cidade não é inferida como província. O CSV admite Shipping Province Name, atributo mapeado ou Shipping Province. Registos existentes no CSV podem receber a província sem criar uma venda duplicada.

A província aparece no cartão compacto, na organização semanal e nos detalhes, entra na pesquisa e nas exportações CSV/PDF, e pode ser editada no formulário. Importação sem valor não apaga uma província já guardada. Pedidos antigos sem o campo precisam de releitura/importação do período; não foi executada uma migração na conta real pelo ambiente de desenvolvimento.

2.7 Mobile, acessibilidade e instalação

Inter local, números tabulares, claro/escuro e alvos de toque geralmente de pelo menos 44 px. Até 1023 px, as quatro áreas principais ficam na barra inferior. Calendários móveis são limitados ao ecrã e diálogos permitem Escape e tratamento de foco. Testar também limites internos dos cartões, telefones indivisíveis e botões legíveis: ausência de scroll na página, sozinha, não prova um layout correto.

A PWA tem manifesto, ícones e service worker. Quando o prompt de instalação não está disponível, a Conta mostra instruções para o navegador/sistema. Não existe APK nativo nem publicação Android/iOS. Internet continua obrigatória para login, consulta e gravação; só a interface pública é guardada pelo service worker.

3. Regras de pedidos, vendas e stock

3.1 Estados e confirmação de venda

Estados armazenados: `pendente`, `remarcado`, `concluido`, `cancelado`.

A regra partilhada `web/sales.mjs:completedSale` aceita receita apenas quando o registo tem status concluido e satisfaz um dos seguintes critérios:

- É um registo manual sem shopifyOrderId; ou
- Foi explicitamente confirmado no painel, com panelConfirmed=true; ou
- Tem shopifyFinancialStatus=paid e shopifyFulfillmentStatus=fulfilled.

Importação Shopify por API/CSV só conclui automaticamente quando pago e processado. Pago sem processado e processado sem pago permanecem pendentes. Cancelamento explícito, refunded completo ou voided são tratados como cancelado pela normalização atual. Esta última regra não substitui um sistema contabilístico de reembolsos; deve ser reavaliada se esse âmbito for implementado.

Registos antigos classificados como concluídos mas sem prova podem aparecer como A verificar e ficar fora da receita. Não alterar todos os estados em massa para melhorar um número apresentado: conferir a fonte e preservar a confirmação explícita válida.

Pedidos distintos são contados por shopifyOrderId; registos manuais são contados por id. Um pedido Shopify pode ter várias linhas de produto, mas conta uma vez no número de pedidos/CPA, origem e entregas por pessoa. Valores e unidades são somados por linha. Custos novos de entrega são por pedido e rateados entre linhas; comissões antigas mantêm a sua distribuição anterior.

3.2 Concluir e Cancelar

Num pedido ligado à Shopify, Concluir:

1. Confirma que o pedido pertence ao UID autenticado e adquire lease Shopify.
2. Lê o estado real; recusa cancelados e pagamentos incompatíveis.
3. Se necessário, cria o processamento a partir dos fulfillment orders suportados.
4. Se necessário e permitido, usa orderMarkAsPaid para registar pagamento recebido.
5. Lê novamente e só confirma localmente quando processado e pago.

Não utiliza captura/cobrança de cartão. Deve ser usado após receber o pagamento. Se o processamento funcionar e o pagamento falhar, a ação pode ser repetida para retomar apenas o que falta. Um retorno HTTP 200 GraphQL, sozinho, não prova sucesso: os userErrors também são verificados.

Cancelar usa orderCancel com refund=false, restock=true e notifyCustomer=false. Não faz reembolso automático; pode repor inventário na Shopify. Isso não sincroniza inventário para o stock do painel.

Enquanto a plataforma não confirmar, a API devolve 202 e o painel mantém o estado anterior, indica atualização e consulta confirmação por GET. O GET nunca pede novo processamento/pagamento/cancelamento à Shopify. A API atualiza todos os registos locais existentes desse order ID, incluindo IDs vindos de CSV/backup, sem criar novas linhas nessa ação.

A edição do formulário de um pedido Shopify não permite mudar o seu estado remoto indiretamente; o campo fica bloqueado e indica Concluir/Cancelar. Sem uma ligação Shopify, registos com ID Shopify podem ser alterados só no painel após confirmação específica desse modo local.

3.3 Reagendamento e datas

Semana por data: dias 1–7 → semana1; 8–14 → semana2; 15–21 → semana3; restantes → semana4. Grupos também podem ser escolhidos manualmente.

Um reagendamento conserva id e valor; guarda dataPedido se ainda não existir; altera data, mes, horario e grupo; define remarcado; acrescenta uma entrada de percurso. As últimas 20 entradas são preservadas. Só pendentes/remarcados podem ser reagendados.

A sincronização preserva campos operacionais locais: horário, entregador, comissão, grupo, mês, descrição, data de entrega, data original e histórico. A data financeira usada nos filtros é **a data atual registada na linha do pedido**, que pode ser a entrega reagendada. Não há um campo contabilístico independente de data de recebimento/conclusão.

3.4 Catálogo e stock

Cada produto local tem id próprio, nome, custo, fotografia, aliases e locais de stock. O vínculo Shopify acrescenta shopifyProductId e shopifyShop, sem substituir custo/foto/quantidades. Novos produtos criados pela ligação começam com stock vazio e custo por definir.

Produtos vinculados têm nome gerido na Shopify; títulos antigos tornam-se aliases. Não são consultados nem escritos campos de inventário Shopify para este vínculo.

O stock local é o total das quantidades introduzidas por local. Não existe um livro de movimentos de stock nem dedução automática garantida em Concluir. Não apresentar stock local como saldo reconciliado da Shopify.

4. Cálculos financeiros e gráficos

A referência financeira é server/model.mjs:calculateMetrics. O frontend calcula séries diárias para desenhar gráficos, que devem permanecer equivalentes ao backend.

Indicador	Regra atual
Receita	Soma de valor das linhas de vendas confirmadas, dentro do período e produto filtrado.
Custo de mercadoria	unitCost capturado, quando existe; caso contrário, custo do catálogo resolvido × quantidade das linhas confirmadas.
Entregas	Taxa capturada por pedido ou valor específico, rateados por linha; comissões legadas mantêm a soma anterior.
Anúncios	Gastos tipo ads atribuídos ao período; campanhas Meta incluídas mesmo sem produto no total geral.
Outros	Gastos tipo outros atribuídos ao período.
Loja	Gastos operacionais únicos na data; recorrentes distribuídos pelos dias do mês. Só no total geral.
Total de custos	Mercadoria + entregas + anúncios + outros + loja.
Lucro estimado	Receita − total de custos.
CPA	Anúncios ÷ pedidos confirmados distintos; null se não houver vendas.
ROAS	Receita ÷ anúncios; null se não houver gasto de anúncios.
ROI	Lucro estimado ÷ total de custos × 100; null se não houver custos.

Exemplo de referência: receita 200 MT, quantidade 2, custo unitário 20 MT, entrega 10 MT, anúncios 40 MT, outros 20 MT e loja 10 MT → custos 120 MT, lucro 80 MT, CPA 40 MT, ROAS 5x e ROI aproximadamente 66,67%.

Um gasto de 70 MT de 10 a 16 de um mês conta 10 MT por dia. O cálculo usa dias inclusivos e sobreposição do intervalo. Um gasto mensal recorrente de 310 MT num mês de 31 dias conta 10 MT por dia a partir da sua data inicial. A recorrência não tem data de fim na implementação atual.

Ao filtrar um produto, os custos gerais da loja ficam fora; gastos gerais sem produto também não são distribuídos arbitrariamente por produtos. O aviso de custo em falta torna o lucro uma estimativa

incompleta, não uma validação de margem.

CPA e ROAS usam todas as vendas confirmadas do período. Não representam atribuição de vendas a campanhas Meta individuais. Os gráficos Acumulado apenas somam a série já calculada; não mudam estes critérios.

Limites financeiros relevantes:

- Vendas concluídas podem guardar `unitCost` e `costCapturedAt`; alterar o catálogo não muda estes custos capturados. Registos antigos sem snapshot usam o catálogo como estimativa; não foi reconstruído o custo real histórico.
- A receita Shopify é preço dos itens × quantidade menos descontos dos itens. Não soma automaticamente frete/impostos nem reconcilia reembolsos parciais.
- Taxa padrão e valor específico são por pedido, mesmo com vários itens. Não somar deliveryFee diretamente nas linhas; usar deliveryAllocations. Comissões antigas continuam por linha até uma alteração explícita no editor.
- effectiveSales prefere linhas canónicas completas da API a resumos CSV do mesmo shopifyOrderId, sem apagar os registos brutos. Uma importação canónica incompleta conserva o resumo legado até completar as linhas conhecidas. IDs determinísticos não resolvem casos sem identidade de pedido, ambiguidades ou todos os cenários de dados legados; conferir os totais reais.

Relatórios usam o período selecionado; séries longas agrupam por semana acima de 62 dias e por mês acima de 180. A consulta atual de meses anteriores usa o intervalo da Lista; o código legado de histórico pode ainda mostrar percursos de meses de origem, mas não é a navegação atual. As taxas de sucesso comparam concluídos com concluídos + cancelados; pendentes não entram nesse denominador.

5. Arquitetura e mapa dos ficheiros

Fluxo principal: navegador → API autenticada Cloudflare Worker → Firestore por HTTPS. O navegador usa Firebase Auth para obter o ID token; esse token acompanha a API em Authorization: Bearer. Tokens Shopify/Meta ficam cifrados no servidor e não são enviados ao frontend.

O Worker também serve os assets da interface e executa Cron. Um backend Express existe para desenvolvimento e testes; parte das rotas e regras é partilhada. Não assumir equivalência completa dos schedulers Node e Worker.

Ficheiro/área	Responsabilidade
painel-negocio-3.html	Estrutura da interface e script clássico principal de pedidos, formulários, calendários, CSV, PDF e gráficos.
web/app.mjs	Firebase Auth, API, estado/baseline, gravações, sessões, integrações e verificação de contagens.
web/navigation.mjs	Composição das quatro áreas e separação Lista/Entregas/Relatórios; encaminha formulários.
web/delivery.mjs	Captura/rateio único da entrega e preservação de snapshots e legado.
web/delivery-settings.mjs	Taxa padrão por negócio e valores específicos no editor.
web/business-tools.mjs	Conta/plano, modo de consulta e interface administrativa.

Ficheiro/área	Responsabilidade
web/privacy-tools.mjs	Exportação, qualidade, aceitação de documentos e encerramento.
web/experience.mjs	Menu de conta, Resumo, diálogos e reagendamentos entre meses.
web/usability.mjs	Simplificação dos formulários, catálogo em diálogo, acessibilidade e composição da interface.
web/experience.css	Tipografia, temas, layout responsivo, cartões e diálogos.
web/orders.mjs	Validação de datas e transformação de reagendamento.
web/sales.mjs	Critério de venda confirmada e contagem distinta de pedidos.
web/order-reports.mjs	Resumos e séries de pedidos, gráficos de relatórios/histórico.
web/catalog.mjs	Resolução de nomes/aliases e campanhas Meta com datas finais.
web/reconcile.mjs	Diálogo de associação de nomes históricos.
web/shopify-catalog.mjs	Interface para selecionar vínculos Shopify/produto local.
web/record-schema.mjs	Validação partilhada dos registos e backups.
web/manual-cleanup.mjs	Compatibilidade para restaurar backups da antiga limpeza; não inclui o antigo botão de apagar tudo.
web/install.mjs	Prompt de instalação PWA e instruções alternativas.
web/sw.template.js	Service worker, cache só de conteúdo público.
server/api.mjs	Rotas partilhadas, autenticação, OAuth, webhooks, métricas e ações dos pedidos.
server/store.mjs	Negócios, persistência, revisões, importação idempotente, leases e cache opcional.
server/model.mjs	Normalização de pedidos/gastos e cálculo financeiro.
server/providers.mjs	GraphQL Shopify, API Meta, paginação, normalização e erros sanitizados.
server/security.mjs	AES-GCM, assinaturas HMAC, domínios, IDs e projeção pública das ligações.
server/access.mjs	Planos, validade, permissões, configuração comercial e versão legal.
server/platform.mjs	Administração, ativações e política de entrega por negócio.
server/privacy.mjs	Exportação de metadados, consentimento e encerramento por lotes.
server/paging.mjs	Cursores, páginas e limites explícitos de processamento.
server/history.mjs	Importação Shopify por período com progresso retomável.
server/catalog.mjs	Vínculo e atualização de nomes Shopify preservando stock/custos locais.
server/status-audit.mjs	Verificação dos estados atuais de pedidos já guardados, incluindo históricos.

Ficheiro/área	Responsabilidade
server/index.mjs, app.mjs, config.mjs, sync.mjs	Backend Node local, configuração e scheduler alternativo.
worker/index.mjs	Entradas fetch/scheduled, runtime, bindings e orçamento de chamadas.
worker/router.mjs	Adaptação das rotas/middleware para Request/Response.
worker/firestore.mjs	Cliente REST Firestore, tipos, consultas e transações.
worker/google.mjs	Identidade de serviço Google e verificação oficial dos utilizadores Firebase.
worker/sync.mjs	Jobs em lotes, cursores, webhooks e alternância entre negócios/provedores.
scripts/build.mjs	Gera dist, bundle, dashboard clássico, CSS, versão e service worker.
scripts/dev.mjs, export-emulators.mjs	Inicialização e snapshots locais dos emuladores.
scripts/wrangler.mjs	Wrapper CLI com configuração/logs locais.
wrangler.jsonc	Worker, assets, cron, variáveis públicas e ambiente local.
firebase.json, firestore.rules	Emuladores e negação de acesso direto ao banco.
public/	Manifesto, ícones, fontes, ajuda e minutas legais.
scripts/backup-account.mjs e restore-account.mjs	Exportação paginada e restauro com prévia e conflitos.
scripts/validate-local.mjs	Orquestração de testes locais com emuladores/Workerd.
scripts/export-docs.py	Geração deste PDF com sumário e navegação.
.github/workflows/validate.yml	Workflow de validação em emuladores, sem credenciais reais.
legacy/	Protótipo anterior, conservado para migração. Não é a aplicação atual.
tests/	Testes unitários, integração, Workerd e criptografia.
dist/	Resultado gerado e ignorado pelo Git; não editar como fonte.

A interface é híbrida: módulos ESM e funções globais do script clássico. web/app.mjs carrega dashboard.js após autenticar/adotar o estado e depois inicializa a experiência. Ao mover elementos DOM, evitar procurá-los por document.getElementById enquanto estão num fragmento desligado do documento.

6. Modelo de dados e concorrência

6.1 Organização Firestore

```
businesses/{uid}
  fluxo/{recordId}
  vendas/{recordId}
  produtos/{recordId}
  ads/{recordId}
  operacional/{recordId}
  connections/shopify
  connections/meta
  leases/{provider}
oauthStates/{randomState}
shopLinks/{canonicalShopDomain}
webhookQueue/{deduplicatedWebhookHash}
platformAudit/{operationHash}
accountClosures/{uid}
system/freeScheduler
```

O documento businesses/{uid} contém ownerId, email, currency=MZN, timezone=Africa/Maputo, createdAt, dataVersion e, quando usados, subscription, activationRequestedAt/Status, legal, accountStatus, usage e deliveryPolicy. O ownerId deve corresponder ao UID autenticado. A dataVersion é incrementada em transações de alteração de dados e invalida o estado/cache.

Cada documento das cinco coleções de registos é um envelope:

```
{
  "record": { "id": "manual_exemplo", "data": "2026-10-13", "...": "campos da coleção" },
  "revision": 1,
  "updatedAt": "2026-10-13T08:00:00.000Z",
  "sourceUpdatedAt": "campo opcional para importações"
}
```

A API projeta o conteúdo de record e acrescenta _revision. Esse campo não pertence ao registo editável; é separado ao guardar. O formato completo autorizado é definido em web/record-schema.mjs, não nos exemplos resumidos deste documento.

6.2 Campos de negócio

Coleção	Campos principais
fluxo	id, data, tipo entrada/saida, valor, descricao, observacao. Preservação do legado.
vendas	id, data, mes, semana, horario, fonte, contacto, local, provincia opcional, descricao, produto, quantidade, valor, delivery, comissao, status.
vendas — origem/prova	shopifyOrderId, shopifyLineItemId, sourceCurrency, panelConfirmed, shopifyFinancialStatus, shopifyFulfillmentStatus.
vendas — custos	unitCost, costCapturedAt; deliveryFeeMode auto/manual, deliveryFee e deliveryFeeCapturedAt.
vendas — importação	shopifyLineCount auxilia a identificação de uma importação canónica completa.
vendas — percurso	dataPedido e reagendamentos[{de,para,semanaAnterior,semanaNova,horario,alteradoEm}].
produtos	id, nome, custo, foto, locais[{id,local,quantidade}], aliases, shopifyProductId, shopifyShop, costConfigured.
ads	id, data, dataFim opcional, tipo ads/outros, nome opcional, produto e valor.
ads — Meta	source=meta, campaignId, campaignName, accountId, sourceCurrency, sourceValue, conversionRate.
operacional	id, nome, valor, data, recorrente booleano.

IDs de linhas API Shopify: sh_{orderId}_{lineItemId}. Produtos novos vinculados: sp_{shopifyProductId}. Gastos Meta: meta_{accountId}_{campaignId}_{date_start}. Não mudar estes formatos sem um plano de migração/idempotência.

Limites de validação relevantes: IDs de até 180 caracteres alfanuméricos/underscore/hífen; texto comum até 4.000 caracteres; registo serializado até aproximadamente 600.000 caracteres; fotografia base64 até 550.000 caracteres; até 50 aliases e 50 locais; até 20 entradas de reagendamento. Números monetários devem ser finitos e não negativos; quantidades por local são inteiros não negativos. O schema atual não exige de forma completa todos os campos de todas as coleções: reforçar obrigatoriedade numa evolução exige compatibilidade/migração de registos antigos.

6.3 Ligações e jobs

Shopify guarda credentials cifradas, shop canónica, connectedAt, lastSync, lastError, progress, salesPolicyVersion, catálogo, statusAudit, historyImport e sinalização de acesso histórico. deliveryPolicy guarda enabled, amount, revision e updatedAt no documento do negócio. Meta guarda credentials cifradas, connectedAt, expiresAt, conta, moeda, câmbio, fuso, lastSync, progress, erros e campanhas não associadas.

statusAudit guarda IDs privados do job, cursor, offset de linhas, prova temporal, contagens, início/fim e permissões históricas. A resposta pública mostra apenas progresso e necessidade de read_all_orders; nunca lista os IDs privados do job ou credentials.

Um lease tem token aleatório e until. A duração atual é 10 minutos. A libertação só apaga o lease se o token ainda corresponder ao dono da execução.

6.4 Gravação e conflitos

O navegador mantém baseline, desired, dirty, busy e failure. Guarda alterações em lotes de até cinco registos, procurando manter o corpo abaixo de cerca de 700 KB. A API aceita até 100 mudanças por chamada, mas o frontend usa lotes menores.

Cada mudança leva id, revision esperada e record completo ou null para apagar. Uma transação lê revisões, recusa alterações obsoletas com 409, aplica mudanças e incrementa dataVersion. Não remover a revisão para contornar um conflito.

Importações ignoram versões de origem mais antigas e evitam regravar registos iguais com o mesmo timestamp. Preservam os campos operacionais definidos. Um cancelamento de origem prevalece sobre a confirmação explícita; uma fonte pendente não deve apagar uma confirmação local válida.

O estado é consultado aproximadamente a cada 60 segundos com a aba visível e ao voltar à aba. ETag tem UID + dataVersion. Uma resposta 304 evita reler coleções. Ligações são consultadas aproximadamente a cada cinco minutos.

No Worker, estado e métricas partilham cache em memória por projeto/ambiente e UID/dataVersion: TTL cinco minutos, até oito MB/20 entradas e limite aproximado de dois MB por entrada. Workers frios voltam ao Firestore. Este cache não é um banco de dados nem substitui as revisões.

7. API e contratos

Todas as rotas /api de dados passam pelo middleware de autenticação, exceto health, config e public-info. O proprietário vem do ID token verificado. Não há parâmetro ownerId aceite do cliente para escolher outro negócio.

Método e rota	Função
GET /api/health	Saúde básica e modo emulator/production; não prova todas as integrações.
GET /api/config	Configuração Firebase Web pública e indicação de emuladores.
GET /api/state	Compatibilidade: cinco coleções com _revision, limitada à capacidade de cálculo; ETag/304.
GET /api/state?manifest=1	UID, versão e coleções para leitura paginada.
GET /api/state/:collection?version =...&after=...&limit=250	Página da coleção da conta, com validação de versão.
POST /api/data/:collection	Gravar/apagar por changes e revisão.
GET /api/metrics?start=...&end=.. .&product=...	Métricas; datas válidas e intervalo limitado a 366 dias.
GET /api/public-info	Identidade/contacto e estado legal/preços/cobrança, sem credenciais.
GET /api/account	Acesso, planos, administração, taxa de entrega e estado de conta/documentos.
POST /api/account/access-request	Pedido idempotente de ativação, sem cobrança.
GET /api/account/export	Metadados da conta e ligações sem segredos; combinar com páginas para exportar registos.

Método e rota	Função
GET /api/data-quality	Cópias ignoradas, custos em falta, revisões pendentes e contagens brutas.
POST /api/account/legal	Aceitação explícita da versão publicada.
GET e POST /api/account/closure	Consultar/start/step do encerramento confirmado.
POST /api/delivery-policy	Política enabled/amount/revision do próprio negócio.
GET /api/admin/businesses	Negócios paginados por administrador autorizado.
POST /api/admin/businesses/:uid/subscription	Ativação manual com revisão/idempotência.
GET /api/admin/audit	Eventos administrativos paginados.
GET /api/admin/health e /api/admin/businesses/:uid/health	Configuração e ligações sanitizadas; não certifica funcionamento externo.
GET /api/connections	Metadados públicos e disponibilidade de configuração.
DELETE /api/connections/:provider	Desligar, preservando registos já importados.
POST /api/sync/:provider	Executar/retomar sincronização; provider shopify ou meta.
POST /api/shopify/orders/:id/status	Solicitar concluido/cancelado e verificar confirmação.
GET /api/shopify/orders/:id/status?status=...	Consultar confirmação sem nova mutação Shopify.
POST /api/shopify/history	start com start/end, step ou cancel; importação histórica retomável.
POST /api/shopify/orders/reconcile	Consultar estados dos pedidos guardados; restart=true inicia nova passagem.
GET /api/shopify/products?after=...	Lista paginada de nomes/IDs para vínculo.
POST /api/shopify/products/link	Vincular produtos com IDs/revisões.
GET /api/meta/accounts	Contas disponíveis para o token da ligação.
POST /api/meta/account	Selecionar conta e conversionRate, iniciar sync.
POST /api/oauth/:provider	Criar tentativa OAuth e devolver URL de autorização.
GET /oauth/:provider/callback	Validar tentativa/assinatura, trocar código e guardar token cifrado.
POST /webhooks/shopify	Verificar HMAC, domínio e deduplicar/guardar job.

Exemplo sintético de gravação:

```
{
  "changes": [
    {"id":"produto_teste","revision":0,"record":{"id":"produto_teste","nome":"Produto teste","custo":2
    0,"foto":"","locais":[]}}
  ]
}
```

A resposta tem revisions:[{id,revision}]. Nas vendas pode incluir também record canónico, com a taxa de entrega capturada pelo servidor; o frontend reconcilia-o com os dados submetidos sem sobrescrever uma edição mais recente ainda em curso. Novo registo usa revisão zero. Apagar usa record:null com a revisão atual.

Códigos relevantes: 400 validação, 401 sessão ausente/inválida ou reautenticação necessária, 402 acesso apenas de consulta, 403 email/permissão, 409 conflito/lease/versão/estado incompatível, 422 limite técnico, 423 conta em encerramento, 428 aceitação legal exigida, 429 excesso/quota, 502 plataforma e 503 configuração/serviço. Existe rate limit por UID de 200 chamadas por minuto, em memória do runtime; não é um rate limiter distribuído global.

Corpos JSON têm limite de um MB; webhook bruto tem limite de cinco MB. Os erros retornam mensagens sanitizadas e, para Firestore, códigos diagnósticos fixos quando reconhecidos.

8. Integração Shopify

8.1 Instalação e OAuth

Aplicação utilizada pelo proprietário: Ponte Painel. O nome comercial não substitui a identificação/configuração efetiva do app no painel Shopify Dev.

Callback de produção: https://painel-shopify.lobojose348.workers.dev/oauth/shopify/callback. Webhook: https://painel-shopify.lobojose348.workers.dev/webhooks/shopify. APP_BASE_URL deve ser o URL HTTPS publicado, sem barra final; a configuração da Shopify deve corresponder exatamente.

Permissões base: read_products, read_orders, write_orders, read_merchant_managed_fulfillment_orders e write_merchant_managed_fulfillment_orders. Aprovações de dados protegidos e casos de locais/prestadores de terceiros podem exigir tratamento adicional.

read_all_orders é solicitado de forma condicional para leitura de históricos, quando a verificação deteta necessidade ou a ligação já foi configurada para esse acesso. A Shopify também deve autorizar esse acesso no app; acrescentar o texto ao OAuth não concede uma aprovação inexistente.

O domínio é original/canónico .myshopify.com, não o domínio público da montra. Aliases diferentes só são aceites quando a identidade Shopify confirma que são a mesma loja. shopLinks impede vincular a mesma loja a dois negócios.

A tentativa OAuth tem state aleatório de 64 caracteres hexadecimais, dono, provider, domínio e expiração de dez minutos. É consumida uma vez em transação. Callback Shopify exige HMAC; não desativar a assinatura para resolver um erro de domínio.

Reautorizar a mesma loja preserva progresso de importação e catálogo. Renovar acesso histórico reinicia a verificação de estados. Uma loja diferente não deve herdar cursores de outra loja. O token é cifrado antes de ir para Firestore.

8.2 Importação e webhooks

A versão API configurada é GraphQL 2026-07. A importação inicial revê cerca de 59 dias. Depois usa alterações desde lastSync, com sobreposição de cinco minutos no Worker. A revisão da política de vendas usa salesPolicyVersion=2 e uma passagem de 59 dias. No Node, a sobreposição incremental continua a ser 24 horas.

Cada página Worker consulta um pedido e grava até cinco linhas por execução. Pedidos com mais linhas continuam pelo seu ID, offset e timestamp. Há proteção contra uma mudança no pedido entre lotes. Pedidos com mais de 100 itens são recusados; paginação adicional desse caso ainda não foi implementada.

Webhooks: orders/create, orders/updated, orders/cancelled e app/uninstalled. A criação de subscrições consulta existentes e reutiliza tópico + callback HTTPS + JSON, sem duplicar indiscriminadamente.

A assinatura usa o corpo bruto recebido. A fila é deduplicada pelo hash de shop + webhook ID. O payload não é a confirmação definitiva: o worker volta a ler o pedido atual na API. A desinstalação remove a ligação/shopLink, mantendo os registos já importados.

8.3 Importação histórica explícita

Pedidos → Relatórios → Importar pedidos criados no período executa server/history.mjs. POST /api/shopify/history recebe action=start, start e end (datas válidas, até 366 dias), depois step; cancel para os lotes futuros sem apagar dados já importados. A consulta usa criação Shopify em UTC; os relatórios locais usam a data atual registada/entrega. Conferir as fronteiras de fuso e reagendamentos antes de comparar totais.

O job guarda intervalo, cursor, ordem/offset e contagens privadas em connections/shopify.historyImport. Consulta um pedido por página e grava até cinco linhas, retomando alterações de timestamp. O navegador acompanha o progresso e o Cron continua. Períodos anteriores à janela normal exigem read_all_orders efetivamente autorizado; uma falha mostra necessidade de reautorização sem inventar cobertura anual. A operação é de leitura Shopify e importação local, não altera pedidos remotos.

Serve também para preencher província em pedidos antigos que a importação anterior não guardou. Custos/snapshots, datas operacionais e valores específicos já guardados são preservados nas regras da store.

8.4 Verificação de contagens antigas

server/status-audit.mjs consulta estados de IDs **já guardados no painel**, sem limitar a leitura à janela incremental de 59 dias. Lê até cinco pedidos Shopify por chamada e atualiza até dez linhas locais, retomando offset quando necessário.

Só consulta estado financeiro, processamento, cancelamento e timestamps. Não cria processamento, não marca pagamento, não cancela na Shopify e não cria pedidos locais novos. Conserva dados operacionais e confirmações explícitas conforme a regra de reconciliação.

Há progresso no relatório e um botão Verificar contagem na Shopify. Ao abrir Relatórios, uma passagem pendente pode ser retomada automaticamente. O Cron também trata a primeira verificação de ligações antigas e jobs incompletos. O navegador atualiza o estado em blocos para reduzir leituras e para ao ficar oculto; o Cron pode continuar.

Pedidos ausentes/inacessíveis são contabilizados como indisponíveis, com os registos locais preservados. O job verifica accessScopes e distingue a falta de read_all_orders de outros casos de indisponibilidade. A reautorização para acesso histórico pede a permissão e permite repetir a passagem.

Importante: esta verificação não importa todos os pedidos em falta de um ano. Um filtro anual mostra apenas os registos que já foram carregados no painel e pertencem àquele intervalo. Não certificar um total anual sem comparar cobertura com a Shopify.

Incidente histórico de referência: o proprietário observou 131 pedidos, 97 cancelados e apenas oito concluídos num filtro anual, apesar de afirmar haver mais pedidos processados e pagos. Foi implementada e testada esta nova verificação, mas **o total real da conta não foi confirmado pelo programador**. Não tratar os números do cenário simulado de testes como resultados da loja.

8.5 Outra loja e troca do aplicativo

Cada conta do painel tem uma única loja e o runtime usa um único par global SHOPIFY_CLIENT_ID/SHOPIFY_CLIENT_SECRET. Outra loja deve usar outra conta no painel para não misturar pedidos, stock, custos ou gastos. Não está implementado um seletor de várias lojas num login nem credenciais de app por cliente.

A distribuição do aplicativo determina que lojas o podem instalar. Apps de uso próprio/distribuição restrita não devem ser apresentados como instaláveis em qualquer cliente. Antes de vender acesso, verificar a organização Partner, distribuição, revisão e acessos necessários com a Shopify. Não basta um token funcionar na loja do proprietário.

Na passagem para uma nova loja: exportar e verificar o backup antigo; desligar a ligação da loja descontinuada sem eliminar o histórico; criar/verificar a conta separada; planear a troca de credenciais globais; configurar e autorizar o novo app; conferir webhooks e leitura. Trocar o Secret afeta HMAC dos webhooks e futuras autorizações do app antigo. Reautorizações não podem ser tratadas como prova de que uma mudança de app global preserva os tokens antigos.

Última informação do proprietário em 05/11/2026: novo Ponte Painel para uso da nova loja criado e versão publicada, sem incorporar no admin e com instalação legada. Naquela confirmação, backup, instalação/testes e troca de Secrets ainda estavam pendentes. O estado atual dessas ações externas deve ser confirmado, não presumido a partir do código. Nunca alterar INTEGRATION_ENCRYPTION_KEY para resolver uma troca de app.

9. Integração Meta Ads

Aplicação utilizada pelo proprietário: Painel do Negócio, caso de uso Marketing API. Callback: https://painel-shopify.lobojose348.workers.dev/oauth/meta/callback. A configuração usa META_API_VERSION=v24.0 e scope ads_read. Verificar suporte/revisão da versão e tipo de login antes de distribuir a clientes externos.

O token fica cifrado no servidor. A reautorização preserva a conta selecionada, moeda, câmbio, lastSync e cursor, substituindo credenciais/expiração. Selecionar outra conta após uma conta já ativa é recusado sem desligar primeiro, para evitar misturar contas inadvertidamente.

A importação consulta Insights por campanha/dia, nos últimos 30 dias, com campaign_id, campaign_name, spend, date_start e date_stop. Não consulta compras para gerar vendas. A janela é revisitada porque a Meta pode corrigir gastos passados. O Worker usa páginas de até cinco resultados e cursor explícito; não segue URLs de paginação arbitrárias fora do domínio oficial.

Se a conta não usar MZN, a taxa é explícita e positiva: 1 unidade da moeda de origem = X MZN. A taxa fica guardada na ligação e em cada gasto. Não há câmbio diário automático. Guardar uma nova taxa pode recalcular despesas reimportadas da janela, logo pode alterar resultados anteriores. A interface repõe a taxa após reload e não a substitui enquanto o utilizador está a editar.

Associação de campanha: prioridade ao nome exato/normalizado do produto ou alias; admite uma data válida no fim, como DD/MM/AA, DD/MM/AAAA ou AAAA-MM-DD, com alguns separadores/parênteses. Não é pesquisa por substring livre. Nomes ambíguos não são associados arbitrariamente. Um conjunto anunciado deve existir como produto conjunto para ser associado por esse nome.

Campanhas sem associação continuam no gasto total geral e são indicadas. O nome original da campanha é preservado. Uma associação nova pode ser usada nos cálculos sem reescrever o gasto bruto.

Uma segunda loja pode reutilizar o aplicativo Meta, mas precisa de autorização na conta separada do painel e seleção da sua conta de anúncios. Usar uma única conta Ads com campanhas de várias lojas

mistura gastos na implementação atual; não há filtro de campanhas por loja nem divisão automática desse cenário. A distribuição/permissões para clientes externos continuam a exigir validação.

A sincronização não elimina gastos manuais semelhantes. Evitar lançar manualmente o mesmo gasto importado. Desligar a integração também não apaga o histórico importado.

10. Segurança, autenticação e privacidade

- Firebase ID token é verificado pelo serviço oficial, incluindo projeto, identidade, validade e revogação. Descodificar um JWT não é verificá-lo.
- Email verificado é obrigatório em produção; o requisito é relaxado apenas nos emuladores.
- O UID autenticado determina o negócio. Consultas/mutações são limitadas aos registos desse negócio.
- firestore.rules nega leitura/escrita direta de todos os documentos pelo cliente. Administração é do backend, que aplica o isolamento.
- Credenciais de integração usam AES-256-GCM: iv, tag e data em base64. INTEGRATION_ENCRYPTION_KEY tem 32 bytes aleatórios, representados em base64. A cifragem deve usar a entrada UTF-8 correta também no runtime Workerd.
- HMAC verifica OAuth Shopify e webhooks. Credenciais, respostas completas das plataformas e URLs OAuth com códigos não devem ir para logs públicos.
- O token Meta usa appsecret_proof nas chamadas relevantes do servidor.
- IDs e domínios são restritos; campos desconhecidos são rejeitados. Datas impossíveis são recusadas. Fotografias são data URLs JPEG/PNG/WebP em base64, não URLs executáveis.
- Headers incluem no-sniff, no-referrer, anti-frame e CSP. A CSP ainda admite unsafe-inline por causa da interface clássica; melhorar isto exige refatoração dos handlers/scripts.
- Service worker não guarda /api, /oauth, /webhooks, tokens ou respostas privadas. A persistência de sessão Firebase é responsabilidade do SDK de autenticação, não uma promessa de cache dos dados do negócio.
- Alterações ainda não guardadas ficam na memória da aba. Fechar uma aba ou perder o processo pode perdê-las; beforeunload apenas avisa.

A chave de cifragem não pode ser trocada sem plano: tokens existentes deixam de ser legíveis. Rotação exige recifrar com a chave antiga ou reautorizar ligações de forma controlada. O JSON da conta de serviço Firebase é um segredo administrativo, não a configuração Firebase Web pública.

Há exportação integral dos registos e metadados públicos da conta, qualidade dos dados, aceitação legal por versão e eliminação em lotes. O início da eliminação exige ELIMINAR, confirmação de perda/backup e reautenticação com auth_time recente (cinco minutos). Um administrador precisa transferir o papel antes de autoeliminar a conta.

O job remove até 25 documentos por lote, incluindo registos, ligações e referências operacionais OAuth/webhooks/auditoria do negócio. O cron continua se o navegador fechar. Um marcador mínimo de UID/estado/datas permanece para impedir recriação por sessões antigas. Ao finalizar, o SDK Firebase elimina o login com reautenticação; se a janela fechar, o utilizador pode entrar e retomar essa etapa. Não apaga pedidos/anúncios externos ou cópias exportadas fora do serviço.

Os documentos públicos em /legal/termos.html, /legal/privacidade.html e /ajuda.html são minutas até aprovação/configuração explícita. LEGAL_VERSION está em server/access.mjs (2026-10-15 nesta

edição). A aceitação guarda versão, momento e UID; ao publicar uma mudança material, atualizar a versão. Aprovação só fica válida com flag, identidade e contacto configurados.

Retenção final, responsabilidades, revisão jurídica, equipas/perfis e callbacks de conformidade das plataformas continuam pendentes. Autoencerramento não substitui customers/data_request, customers/redact, shop/redact e os fluxos Meta exigidos para distribuição. Não apontar esses callbacks para rotas que apenas devolvam sucesso sem processar a operação nem apresentar o sistema como certificado.

11. Configuração e ambientes

11.1 Identificadores e variáveis públicas

Nome	Produção atual/referência
Worker	painel-shopify
FIREBASE_PROJECT_ID	painel-cash-on-delivery
FIREBASE_AUTH_DOMAIN	painel-cash-on-delivery.firebaseapp.com
FIREBASE_WEB_API_KEY	Configuração pública em wrangler.jsonc e config/firebase.production.json; não repetir aqui.
APP_BASE_URL	https://painel-shopify.lobojose348.workers.dev
USE_FIREBASE_EMULATORS	false em produção; true apenas localmente.
SHOPIFY_API_VERSION	2026-07
META_API_VERSION	v24.0
compatibility_date	2026-10-12
compatibility_flags	nodejs_compat

11.2 Secrets

Nome	Conteúdo/função
FIREBASE_SERVICE_ACCOUNT	JSON completo da identidade Google do projeto correto, só no Worker.
INTEGRATION_ENCRYPTION_KEY	Chave de 32 bytes em base64, necessária para ler tokens cifrados existentes.
SHOPIFY_CLIENT_ID	ID do app; pode ser público, mas deve permanecer disponível na configuração do deploy.
SHOPIFY_CLIENT_SECRET	Segredo do app Shopify para OAuth/HMAC.
META_APP_ID	ID do app Meta, disponível na configuração.
META_APP_SECRET	Segredo do app Meta.

IDs podem ser variáveis normais, mas foram também tratados como Secrets no processo de configuração do proprietário. Não apagar uma binding assumindo que um ID está em wrangler.jsonc. O código não inclui valores secretos. Não usar dumps de ambiente para verificar configuração; verificar nomes/presença e testar operação segura.

No Node, a identidade Google usa Firebase Admin/Application Default Credentials; num cenário real, verificar credenciais e permissões IAM sem colocar ficheiros privados no Git. .env.example é exemplo local; config/production.env.example é referência de variáveis de produção. Estes ficheiros não comprovam que os Secrets foram instalados.

Para gerar uma chave nova num ambiente confiável:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

O resultado é secreto. Guardar no gestor de Secrets; não colar em chat, código fonte ou documentação. Não gerar uma chave nova para substituir a existente de uma instalação com tokens cifrados sem plano de recuperação.

11.3 Administração e ativação comercial

PLATFORM_ADMIN_UIDS é a lista de UIDs administrativos separados por vírgula, configurada no servidor/Cloudflare; pode ser guardada como Secret. O primeiro registo e um email enviado pelo cliente não dão administração. Não publicar valores pessoais de UID ou credenciais neste manual.

SUBSCRIPTION_ENFORCEMENT=false (ou ausente) mantém o acesso piloto. OPERATOR_NAME e SUPPORT_EMAIL identificam o operador; LEGAL_DOCUMENTS_APPROVED=false mantém as minutas sem exigência de aceitação. Os modelos .env.example e config/production.env.example não ativam a fase comercial. Não substituir estes valores sem um plano de transição para contas existentes.

O proprietário informou ter configurado o UID administrador no Cloudflare. Confirmar o botão Administração com a sessão correta; não inferir a configuração efetiva a partir do repositório. Pagamentos e bloqueios foram deixados como última fase por decisão do proprietário, priorizando a ligação simples de clientes à Shopify/Meta.

12. Desenvolvimento, compilação e publicação

12.1 Arranque local

Requisitos: Node >=22, npm compatível com o lockfile e Java 21 para Firestore Emulator. Use o checkout existente; não é preciso um worktree extra.

```
npm ci
npm run dev
```

npm run dev compila a interface, inicia Auth/Firestore emulados no projeto demo-painel, aguarda disponibilidade e inicia Node na porta 3000. Auth usa 9099; Firestore usa 8080. Não ligar emuladores a um projeto real. Criar contas e registos sintéticos para desenvolvimento.

O encerramento normal exporta snapshots para .emulator-data; ficheiros auxiliares/snapshots anteriores ficam em .local. O encerramento abrupto pode perder alterações locais desde a última exportação. Não apagar esses diretórios para resolver um problema de interface.

Com os emuladores já ativos, num segundo terminal:

```
npm run dev:worker
```

Worker local usa 8787 e o ambiente local do Wrangler. Alternativa: iniciar npm run emulators separadamente e depois dev:worker, sem arrancar uma segunda cópia nas mesmas portas.

Não abrir o HTML por file:// nem servir só os ficheiros estáticos: autenticação e persistência dependem do backend.

12.2 Build

```
npm run build
```

scripts/build.mjs extrai o último script clássico do HTML para dist/assets/dashboard.js, gera bundle ESM de web/app.mjs com esbuild, copia CSS/fonts/ícones, gera HTML e service worker. Chart.js e jsPDF estão no bundle, sem dependência de CDN em runtime.

A versão painel-build é um hash de 12 caracteres do HTML, app e ficheiros listados em extra no build. **Ao acrescentar um módulo ou alterar regras relevantes para a versão publicada, conferir a lista usada no hash.** Preservar a posição do CSS usada pelo script; não editar dist como fonte.

Dependências fixadas no package.json/lockfile: Firebase Web 13.0.0, firebase-admin 14.5.0, Express 5.3.0, Chart.js 4.4.4, jsPDF 4.2.1, esbuild 0.28.2, firebase-tools 15.33.0 e Wrangler 4.149.0. Atualizações exigem testes, sobretudo compatibilidade Worker, cifragem e APIs das plataformas.

12.3 Publicação

Cloudflare usa o repositório GitHub e branch main. Configuração de referência: raiz do repositório, build npm run build e deploy npx wrangler deploy --env="". Não publicar como Pages estático: as rotas de servidor são essenciais.

```
npm run check:worker
```

É dry-run, não deploy. Publicação direta, quando o operador estiver autenticado e autorizado:

```
npm run deploy:worker
```

No fluxo utilizado, alterações testadas são commitadas e enviadas para main; a Cloudflare publica automaticamente. O checkout de desenvolvimento esteve na branch cloudflare-free, com push HEAD:main. Esse nome local não muda a branch que a Cloudflare acompanha.

Verificar que o commit chegou à branch certa e comparar meta painel-build do site com dist/index.html. Esse teste confirma a versão de assets publicada; não prova o estado das credenciais, os totais dos clientes ou o funcionamento de cada ação real.

Confirmar que APP_BASE_URL, callbacks, domínio autorizado Firebase e Secrets correspondem ao novo endereço. Não remover Secrets existentes ao atualizar variáveis. Não ativar Workers Paid/Blaze automaticamente.

13. Testes e critérios de aceitação

Com emuladores demo-painel ativos em 9099/8080 e as dependências instaladas:

```
npm test
npm run test:integration
npm run test:commercial
npm run test:recovery
npm run test:delivery
pip install playwright==1.62.0
python -m playwright install chromium
npm run test:browser
npm run dev:worker
# Noutro terminal, após a saúde local responder mode=emulator:
npm run test:worker
npm run test:worker:crypto
npm run check:worker
```

Integração e navegador usam Auth/Firestore emulados e provedores simulados. Worker integration exige Workerd local em 8787. Worker crypto inicia fixture própria. Não apontar estas suites para produção. Node >=22 e Java 21 são necessários; o emulador usa somente projeto demo-.

scripts/validate-local.mjs reúne integração, comercial, recuperação, entrega, navegador e Workerd/crypto. Requer emuladores já iniciados e a porta 8787 livre; inicia/termina apenas o seu Worker. .github/workflows/validate.yml prepara Node 22, Java 21, Python e Chromium e executa validação nos emuladores. A existência do workflow não comprova uma execução remota bem-sucedida.

Suite	Evidência na última implementação, código 53fa8d7
Unitários	56 testes passaram: segurança, normalização, moedas, província, vendas, aliases, custos, datas, relatórios, snapshots/rateio de entrega.
Integração Node	25 cenários passaram: isolamento, revisões, OAuth, importações, ações, reconciliação e catálogo com provedores simulados.
Comercial	9 cenários passaram: papéis, validade, aceitação legal, paginação maior que 500, conta/exportação/encerramento e ausência de mutações remotas sem acesso.
Recuperação	2 cenários passaram: histórico por lotes e backup/restauro privado com prévia, identidade, repetição e conflito.
Entrega	Cenário API passou: política isolada/revisão, vários itens, valor zero, alterações/sync, exportação e bloqueio de edição.
Navegador	Suite versionada tests/browser-commercial.py/.mjs passou: Conta, administração, exportação/eliminação, navegação, província, taxa padrão/valor específico e UI móvel.
Workerd	18 verificações passaram: API/Firestore REST, cursores, mais de 500 registos, jobs, auditoria, taxa capturada e encerramento.
Crypto/dry-run	Cifragem Workerd foi validada nas intervenções anteriores; build e dry-run da implementação de entrega passaram. Dry-run não publica nem certifica CPU/quota real.

Estes resultados são evidência da implementação, não uma nova execução de todas as suites durante a edição deste documento. Não certificam totais reais da conta, revisão das plataformas ou a execução atual do GitHub Actions.

No navegador, validar 320/360/390/412/768/1280 px e dados com nomes compridos, vários produtos e dois números de telefone. Verificar também limites internos dos cartões, legibilidade dos botões e telefones inteiros. A suite atual contém estes percursos; scripts temporários de auditorias antigas são apenas evidência histórica complementar. Testar temas, dispositivos físicos e navegadores relevantes antes de ampliar clientes.

Critérios mínimos:

- Dois UIDs não leem nem alteram dados um do outro; administrador vem da configuração do servidor.
- Sessão, falha de dados, quota, vazio, conflito e progresso apresentam estados coerentes.
- Revisão antiga falha sem apagar uma gravação mais recente; leitura paginada não mistura versões.
- Reimportação não duplica linhas/gastos; cópias CSV/API são tratadas sem apagar originais.

- Reagendamento atravessa meses e sobrevive à sincronização com ID/data original/percurso.
- Pago sem processado e processado sem pago não geram conclusão automática; Concluir confirma ambos; polling não repete mutações.
- Auditoria lê estados já guardados; histórico importa pedidos em falta; indisponibilidade não significa cancelamento.
- Compras Meta não geram vendas; nomes ambíguos não recebem associação arbitrária; câmbio sobrevive a reload/reautorização.
- Custo/foto/stock local sobrevivem ao vínculo Shopify; província não é inventada a partir de cidade.
- Taxa conta uma vez por pedido e fecha os cêntimos entre produtos; pendentes/cancelados ficam fora; zero manual e snapshots são respeitados.
- Mudar o padrão ou editar só o contacto não altera custos históricos; restauro conserva snapshots.
- Expirado conserva consulta/exportação, mas não grava nem provoca operações Shopify; encerramento é confirmado e retomável.
- Backups inválidos/conflituosos são recusados antes de substituir registos existentes.

Ensaiar operações com efeitos em contas/pedidos de teste e autorização adequada. Nenhuma destas suites executa Concluir/Cancelar em clientes reais.

14. Operação e diagnóstico

14.1 Agendamento e quotas

Cron de produção: uma execução por minuto. Prioridade para um encerramento de conta pendente, depois para um webhook pendente; sem webhook, alterna Shopify/Meta de um negócio e avança para o seguinte. Uma importação histórica pendente antecede auditoria/catálogo/importação incremental Shopify. Webhooks de contas sem acesso de edição podem ficar pausados e retomar após renovação. Contas encerradas não voltam a importar. Catálogo vinculado é reconciliado diariamente em lotes; falhas recebem estado/backoff para não bloquear pedidos indefinidamente. Uma janela de sincronização concluída tem intervalo mínimo de cerca de 15 minutos.

Isto não significa que todos os pedidos de todos os negócios sincronizam a cada minuto. Vários negócios/lotes ou uma fila grande de webhooks podem causar atrasos e atrasar a Meta. O Node local usa timers de cinco segundos para fila e cerca de 15 minutos para sync, com outras dimensões de lote; não reproduz automaticamente toda a política do Worker.

O runtime limita chamadas controladas a 40 por invocação, reservando margem para a plataforma. Valores comerciais de quota/preço mudam; verificar as páginas oficiais antes de dimensionar. Dry-run/local não certifica a CPU efetiva do Workers Free.

O estado carrega por manifesto e páginas de até 250 registos com verificação de versão. Consultas internas completas usam páginas de 500, sem truncar. O cálculo desta instalação tem limite técnico de 10.000 registos somados e orçamento de 28 páginas internas. A exportação paginada permanece disponível. Isto não amplia as quotas gratuitas do Firebase nem garante desempenho para qualquer volume.

Firebase Spark já atingiu quota de leituras durante a utilização. Cache, ETag e sobreposição menor foram introduzidos para reduzir consumo, não para repor quota esgotada. Quotas contam documentos, não só chamadas HTTP. Consultar Usage e os logs antes de atribuir um erro ao login. Não autorizar upgrades por omissão.

14.2 Falhas comuns

Sintoma/código	Conferir e agir
FIREBASE_QUOTA	Usage Firestore, leitores/sync, quota e reset aplicável. Reduzir consumo; manter plano autorizado.
FIREBASE_ACCESS	Permissões IAM da conta de serviço e projeto correto; não abrir regras do cliente.
FIREBASE_IDENTITY	JSON/identidade recusada, projeto, chave e disponibilidade do serviço Google.
FIREBASE_UNAVAILABLE / TIMEOUT	Disponibilidade/rede/latência; repetir de forma controlada.
Sessão reconhecida, dados não carregam	GET /state e páginas, código de diagnóstico, quota e limite técnico; usar recuperação de sessão.
Integração precisa de administrador	Presença de ID/Secret/chave e APP_BASE_URL em produção; rede/ambiente correto.
Autorização expirada/inválida	Nova tentativa OAuth do painel; state expira/é consumido uma vez. Não recarregar callback antigo.
Domínio Shopify divergente	Domínio .myshopify.com correto e validação da identidade; não retirar HMAC.
Falha de INTEGRATION_ENCRYPTION_KEY	Base64 de 32 bytes, Secret correto, cifragem UTF-8, preservação da chave anterior.
Shopify GraphQL ACCESS_DENIED em HTTP 200	Escopos, autorização de dados protegidos e campos consultados; HTTP sozinho não prova sucesso.
Totais baixos num filtro anual	Cobertura dos registos carregados, data registada/entrega, estados/provas e statusAudit. Comparar com Shopify.
Pedido antigo indisponível	accessScopes/read_all_orders ou pedido ausente; não reinterpretar ausência como cancelado.
Concluir processa mas não paga	canMarkAsPaid/userErrors; mostrar falha e retomar apenas o estado em falta.
Campanha sem produto	Nome/alias, data final válida, conjunto cadastrado e ambiguidades.
Câmbio inesperado	Taxa guardada/conta/moeda, reimportação da janela e histórico da alteração da taxa.
409 ao guardar	Conflito de revisão ou lease; preservar dados locais e conciliar, não forçar revisão zero.
Atualização não aparece	Commit/main, deployment e painel-build, service worker/versão, API e jobs pendentes.

Logs devem mostrar códigos/etapas seguros, nunca tokens, chaves, Authorization, payloads privados completos ou clientes. Erros de provider são sanitizados. O operador deve recolher apenas a evidência mínima necessária para diagnóstico.

A última correção de contagens não valida a conta real por si. Exigir mensagem de fim/progresso, número de indisponíveis e comparação com a Shopify antes de dizer que o total de vendas do

proprietário está correto.

15. Backups, migrações e recuperação

A exportação normal contém os registos das cinco coleções. A exportação completa também inclui metadados da conta e ligações sanitizadas; ambos os percursos estão descritos nesta edição. Não inclui tokens de integração nem constitui um backup administrativo integral de Firestore/Auth. Não permite clonar autenticação, Secrets, jobs, shopLinks ou a infraestrutura.

Importação normal exige conta sem dados, tamanho até dez MB, formato reconhecido e validação antes de guardar. Conserva a cópia original. Produtos legados com estoque/local são transformados em locais. Campos/IDs/datas/fotos inválidos e IDs repetidos são recusados.

Apesar da validação antecipada, a importação completa grava vários lotes: não é uma transação atómica única de todo o negócio. Uma falha de rede/conflito a meio pode exigir recuperação assistida. Não insistir numa segunda importação indiscriminada numa conta parcialmente preenchida.

A restauração dos backups da antiga limpeza só repõe registos manuais em falta, valida antes de inserir, rejeita duplicações e não substitui IDs existentes ou registos de integração. A limpeza em massa em si não existe mais.

Dados antigos de localStorage só são legíveis no mesmo domínio/navegador em que foram guardados. Usar a versão legacy para exportar se necessário; não presumir que o domínio workers.dev consegue ler outro domínio.

Recuperação de uma instalação completa exige também acesso seguro a Firebase Auth/Firestore, Secrets, apps Shopify/Meta e configuração Cloudflare. Manter uma cópia segura da chave de cifragem e plano de restauração do banco compatível com orçamento/permissões; o projeto não implementa backup automático integral de produção.

Antes de migrar ou reescrever: exportar dados, mapear IDs e aliases, confirmar datas/grupos, preservar revisions/sourceUpdatedAt ou estabelecer uma nova política explícita, inventariar ligações sem copiar tokens para ficheiros públicos e testar uma restauração de exemplo.

16. Limitações e trabalho pendente

Prioridades técnicas e funcionais:

1. Medir capacidade de paginação e cálculos no volume real antes de ampliar a base de clientes; não apresentar totais parciais como completos.
2. Validar cobertura histórica real com o proprietário. A importação por período e read_all_orders já existe; statusAudit revê apenas IDs guardados. Não confundir as operações nem certificar cobertura anual sem comparação.
3. Concluir validação da contagem real reportada pelo proprietário após a nova verificação/autorizações. Não substituir essa validação por testes simulados.
4. Modelar pedido e linhas de forma explícita para evitar contagens mistas, comissões duplicadas e coexistência de CSV/API. Deduplicação deve preservar histórico e valores legítimos.
5. Snapshots de custos estão preparados para novas conclusões. Continuam pendentes eventos/data própria de pagamento/conclusão e regras integrais de frete/impostos/reembolsos para resultados contabilísticos.
6. Criar movimentos de stock se for pretendida dedução automática, sempre independente da Shopify conforme a decisão de produto.

7. Melhorar justiça/prioridade do scheduler, retries/backoff, limpeza de OAuth/jobs antigos e reprocessamento administrativo de jobs falhados.
8. Ampliar a suite de navegador versionada e validar dispositivos físicos, temas e navegadores relevantes; não depender apenas de medições de largura da página.
9. Migrar gradualmente o script clássico/handlers inline para componentes/módulos, mantendo contratos e acessibilidade; tornar CSP mais restritiva.
10. Aprovar retenção e documentos e implementar endpoints de conformidade específicos das plataformas; o autoencerramento/exportação já está preparado.
11. Planos, validade, direitos de acesso e ativação administrativa foram implementados, mas ficam desligados por padrão. Definir preços, pagamentos/recibos, suporte, capacidade e distribuição dos apps antes de ativar a fase comercial. Não há cobrança automática.
12. Se necessário, suportar outras moedas Shopify, câmbio por data, múltiplas contas Ads e atribuição de campanhas; nenhuma dessas capacidades deve ser presumida pela existência de um filtro por produto.

O CSV de meses anteriores usa a data atual do registo, enquanto a coluna de data no histórico pode mostrar a origem do reagendamento. Clarificar/exportar ambas as datas é uma melhoria pendente; não usar essa diferença como prova de que o registo foi duplicado.

Outras lacunas: mais de 100 itens Shopify; fulfilment de certos terceiros; histórico maior que a janela Meta; fotos no Firestore em vez de armazenamento dedicado; reconciliação de SKU/variantes em catálogo centrado em nomes/IDs de produto; fusos e corte diário Shopify/Meta não reconciliados contabilisticamente; falta de auditoria de ações de negócio completa.

17. Como reconstruir do zero

Uma reconstrução pode trocar a stack, mas deve conservar as decisões de produto e migrar os dados com prova de equivalência. A implementação atual é uma referência, não uma obrigação de manter um HTML monolítico.

Etapa 1 — Especificação e dados

Ler este manual com o proprietário. Confirmar moeda, fuso, definição de venda, datas dos relatórios, custos, comissão por linha/pedido, stock independente, retenção e dimensão do histórico. Inventariar os registos existentes e duplicações sem apagar nada. Guardar backups privados e um plano de reversão.

Etapa 2 — Identidade e isolamento

Escolher identidade e banco compatíveis com o orçamento. Implementar UID/negócio, API autenticada, email verificado, isolamento e revisões. Testar dois utilizadores antes de importar dados reais. Manter credenciais de plataforma exclusivamente no servidor.

Etapa 3 — Domínio e funções puras

Implementar modelo pedido/linha/produto/gasto, critérios de confirmação, aliases, rateio de gastos e fórmulas. Criar os cenários da secção de testes. Reproduzir snapshots de mercadoria/entrega existentes e só acrescentar data própria de recebimento numa evolução autorizada; não mudar históricos silenciosamente.

Etapa 4 — Interface móvel primeiro

Construir Resumo, Pedidos, Produtos e Resultados; separar Lista, Entregas e Relatórios, com cartões móveis compactos, menu de conta, província, filtros, detalhes, diálogos e reagendamento. O mesmo filtro deve controlar indicadores, gráficos e lista. Incluir estados de carregamento, vazio, falha, conflito e progresso. Preservar Inter/números legíveis ou uma alteração aprovada pelo proprietário.

Etapa 5 — Shopify

OAuth/HMAC, domínio/loja canónica, token cifrado, webhooks assinados/deduplicados, importação paginada/idempotente e jobs retomáveis. Validar leitura antes de escrever. Depois implementar Concluir processado + pago, Cancelar sem refund, confirmação assíncrona e consulta de históricos autorizados.

Etapa 6 — Meta

OAuth ads_read, seleção de conta, câmbio explícito, importação por campanha/dia e IDs estáveis. Não criar vendas a partir de compras Meta. Resolver nomes/datas sem associações arbitrárias e preservar campanhas sem produto no total.

Etapa 7 — Catálogo e inventário local

Vincular IDs/títulos Shopify ao produto local, guardar aliases de renomeação e manter custo/foto/stock independente. Não transferir inventário Shopify. Se for necessário livro de stock, implementar como domínio próprio com migração explícita dos saldos locais.

Etapa 8 — Migração e comparação

Importar cópia num ambiente isolado. Comparar por pedido/linha/produto/período estados, valores, unidades, custos capturados, taxa por pedido/comissões legadas, província, aliases e gastos. Tratar de forma explícita duplicates CSV/API, datas reagendadas e provas antigas. Comparar contas totais e amostras com as plataformas; não apenas aparência dos gráficos.

Etapa 9 — Operação e publicação

Configurar orçamento/quotas, monitorização, retries, backups, rollback, callbacks e Secrets. Publicar uma versão de teste, validar UID/isolamento e fluxos em contas sintéticas. Fazer passagem controlada de dados e validar números reais com o proprietário antes de desativar a versão anterior.

Etapa 10 — Continuidade

Documentar versões, variáveis, contratos, migrações e diferenças em relação a este manual. Manter decisões confirmadas separadas de propostas e atualizar testes/documentação no mesmo trabalho que muda uma regra.

18. Lista de passagem ao próximo programador

O proprietário deve conceder acesso apropriado às seguintes áreas, sem enviar Secrets em chat/documentos:

- Repositório GitHub e branch de produção.
- Conta/projeto Cloudflare, deployments, bindings e logs.
- Projeto Firebase correto, Auth, Firestore, Usage, regras e IAM.
- App Shopify, distribuição, versões, permissões e loja de teste.

- App Meta, caso de uso, permissões, revisão, conta de anúncios de teste e fuso/moeda.
- Gestor/cópia segura dos Secrets, sobretudo a chave de cifragem existente.
- Backups privados de dados e critérios de retenção/recuperação.

Na primeira sessão de trabalho:

1. Confirmar branch/commit, ler schema e regras de venda e verificar documentação contra código.
2. Executar npm ci, ambiente emulado, testes e build/dry-run.
3. Conhecer a versão publicada por painel-build e verificar modo production, sem revelar Secrets.
4. Confirmar a cobertura e os estados reais dos pedidos do proprietário, especialmente o incidente anual ainda não validado.
5. Levantar dimensão das coleções/quotas antes de aumentar importação histórica.
6. Não usar pedidos de clientes para ensaios de Concluir/Cancelar nem desativar verificações para desbloquear um teste.
7. Registar qualquer mudança de regra com migração e validação do seu efeito nos históricos.

Este manual e a cópia PDF são uma fotografia do código indicado no início. A versão Markdown é a referência editável; atualizar e regenerar a cópia PDF quando houver alterações substanciais.

19. Taxa de entrega padrão e exceções

Cada negócio pode definir o custo fixo combinado com o entregador em Conta → Taxa de entrega padrão. Existe um atalho na secção de custos de Resultados. A opção começa desligada, sem qualquer valor imposto ao proprietário.

A taxa aplica-se uma vez por pedido concluído, mesmo que contenha vários produtos. Não é um pagamento automático ao entregador nem altera o frete cobrado na Shopify. Pedidos pendentes e cancelados não entram no custo. Um valor específico por pedido, incluindo zero, pode ser definido no formulário de edição e tem prioridade sobre o padrão.

Histórico e exceções

O servidor captura a taxa na conclusão. Alterar ou desligar o padrão não recalcula custos de pedidos já concluídos e guardados. Nos pedidos importados pela primeira vez já concluídos, a taxa só é aplicada quando a data original do pedido não é anterior ao dia de ativação/configuração vigente, no fuso de Maputo; não foi inventada uma data real de entrega que a Shopify não forneceu. Pedidos antigos podem receber um valor explícito pelo editor.

Valores antigos em `comissao` continuam com os seus cálculos anteriores por linha. O editor apresenta “Manter custo já registado”; só muda o modelo desse pedido quando o utilizador escolhe explicitamente o padrão ou um valor específico. Uma edição apenas do contacto não deve alterar custos históricos.

Os custos automáticos/específicos de um pedido são repartidos pelos produtos proporcionalmente ao valor de cada linha. Sem receita positiva, usa-se a quantidade. O rateio distribui os cêntimos restantes para garantir que a soma das linhas coincide com a taxa. Valores antigos de comissão mantêm a sua distribuição histórica por linha.

Implementação

- `businesses/{uid}.deliveryPolicy`: enabled, amount, revision e updatedAt. O endpoint autenticado `POST /api/delivery-policy` exige permissão de edição e revisão atual; aceita valores de zero a 1.000.000 MT, com até duas casas decimais. Não modifica negócios de outros UIDs.
- Vendas: `deliveryFeeMode` auto/manual, `deliveryFee` como valor integral por pedido e `deliveryFeeCapturedAt`. O snapshot é repetido nas linhas; **não somar esse campo diretamente**.
- `web/delivery.mjs`: captura e rateio partilhados pelo servidor e pelos gráficos. Em importações por lotes, a captura mais antiga conhecida do pedido determina o padrão. Um valor manual tem prioridade.
- `server/store.mjs`: captura no servidor para alterações locais, conclusões Shopify e importações. A API devolve o registo canónico após gravar, para atualizar os gráficos locais com a taxa realmente guardada.
- `web/delivery-settings.mjs`: configuração por negócio e escolha entre padrão, valor específico e valores legados no editor.
- Restauros de registos concluídos sem modo de entrega conservam o legado. Novos registos manuais usam o modo auto explicitamente; snapshots já exportados são preservados, sem criar uma taxa nova ao restaurar.
- Métricas, gráficos diários, totais por entregador e exportações CSV usam o mesmo rateio. Não há desconto automático de stock.

Backup e testes

Os snapshots de custo integram os registos exportados e são preservados ao restaurar. A configuração do padrão também aparece nos metadados da exportação completa da conta; os scripts de restauro recuperam os registos, sem reativar configurações da conta. Confirmar/configurar o padrão no destino após uma migração.

Testes: `npm test`, `npm run test:delivery` (API/emuladores), `npm run test:browser` e `npm run test:worker`. Cobrem pedidos com vários itens, rateio por produto, alteração de taxa sem recalcular conclusões, valor específico zero, cancelados/pendentes, revisões concorrentes, isolamento por conta, sincronização, restauro e UI móvel. Os testes usam apenas contas sintéticas.

20. Preparação comercial, administração e operação

20.1 Estado e decisões

A base de acesso comercial existe: planos propostos Piloto, Essencial e Profissional, validade, teste autorizado, suspensão, pedido de ativação, idempotência e registo administrativo. Todos os preços do catálogo de planos estão por definir; os nomes não prometem funcionalidades/quotas diferentes. Não há gateway, verificação automática de recebimento, emissão fiscal ou cobrança automática.

O estado piloto significa apenas que o bloqueio por assinatura está desligado. Uma conta verificada pode utilizar as funcionalidades enquanto SUBSCRIPTION_ENFORCEMENT estiver false/ausente. O proprietário decidiu deixar pagamentos e bloqueios para a última fase, priorizando ligação simples das lojas e anúncios. Não ativar restrições por iniciativa do programador.

Quando autorizado o controlo comercial, pendente/suspenso/expirado fica em consulta, com leitura e exportação preservadas. Edição, sincronização e ações Shopify exigem acesso ativo; OAuth/callback e

tarefas de fundo também fazem verificações de acesso. Pausar acesso não é apagar dados. A taxa de entrega é um custo do negócio e não tem relação com o preço do plano vendido.

20.2 Administração e recebimentos manuais

Só UIDs em PLATFORM_ADMIN_UIDS recebem administração. A área lista negócios por páginas, permite gerir plano/estado/validade, mostra pedidos de ativação e diagnóstico sanitizado das ligações e consulta platformAudit. Não concede colaboradores acesso a dados de outra loja.

A ativação recebe requestId, revision, planId, status, expiresOn, method e, opcionalmente, reference/amount. Status: active, trial ou suspended; method: manual, mpesa, emola ou bank. São registos administrativos de recebimento conferido fora do painel; escolher M-Pesa ou e-Mola não integra automaticamente esses serviços. Validade termina no fim do dia de Maputo e a API limita a três anos no futuro. Alterações obsoletas e reutilização incompatível de requestId falham com 409.

Não ativar acesso pago com base apenas numa captura de ecrã de um pagamento. Conferir o recebimento no meio autorizado pelo vendedor e registar a referência, sem divulgar credenciais. Definir com o proprietário preços, duração, renovação, cancelamento, reembolso, recibos/impostos e suporte. Nenhum valor comercial foi escolhido pelo código.

20.3 Sequência de abertura ao público

1. Exportar a conta e ensaiar recuperação. Conferir números reais e cobertura com Shopify/Meta, custos, câmbio e múltiplos itens.
2. Confirmar a conta administrativa, definir suporte, preços e meios de recebimento; preparar acessos existentes sem bloquear ninguém.
3. Validar distribuição Shopify/Meta para clientes externos, revisão/permissões, dados protegidos e callbacks de privacidade. A autorização da conta do proprietário não comprova distribuição comercial.
4. Rever identidade do vendedor, contratos, privacidade, retenção, fornecedores e responsabilidades. Configurar OPERATOR_NAME e SUPPORT_EMAIL; só depois de aprovação explícita usar LEGAL_DOCUMENTS_APPROVED=true. Atualizar LEGAL_VERSION quando necessário.
5. Ativar previamente contas existentes que devam continuar a editar, com validade futura, e testar criação de conta, autorização das plataformas, importação e uso completo com uma conta externa autorizada.
6. Apenas quando o proprietário decidir, ativar SUBSCRIPTION_ENFORCEMENT=true e conferir renovação/expiração. Fazer abertura limitada com medição de quotas e suporte próximo antes de ampliar a base.

Voltar a SUBSCRIPTION_ENFORCEMENT=false libera novamente edição às contas verificadas; não é um mecanismo de renovação individual. Para transferir administração, adicionar o novo UID, testar a sessão e só depois retirar o anterior. Não reconfigurar Secrets ou faturação de infraestrutura como atalho de diagnóstico.

20.4 Pendências externas e capacidade

Verificar organização/tipo/distribuição do app Shopify e eventual organização Partner com distribuição pública/revisão para outras lojas. Confirmar acesso histórico, dados protegidos e cenários de fulfillment suportados. Na Meta, verificar login, ads_read, funções de teste, acesso externo, revisão/verificação empresarial e validade de tokens conforme o caso. Não foi submetida uma aprovação de distribuição por este manual.

Ainda faltam os contratos de conformidade específicos, incluindo customers/data_request, customers/redact, shop/redact e os fluxos de eliminação/revogação Meta aplicáveis. O autoencerramento implementado não os substitui. Não prometer conformidade certificada.

A configuração gratuita tem quotas reais. O limite técnico de 10.000 registos por negócio e o orçamento de 28 páginas não são um plano comercial nem garantia de desempenho/capacidade. Novos negócios têm usage.counts transacionais; legados sem contadores continuam sujeitos ao limite de cálculo/leitura, sem inventar contagens. Paginação e cache não repõem quota nem garantem baixa latência em várias empresas simultâneas. Medir filas, atrasos, CPU, fotos, leituras/escritas e tráfego antes de prometer disponibilidade.

20.5 Exportação completa e restauro operacional

Conta → Dados oferece exportação dos registos com metadados da própria conta, sem Secrets/tokens. A rota /account/export fornece metadados; os registos vêm de todas as páginas do estado, com verificação da versão no fim. Também há diagnóstico de cópias ignoradas, custos em falta, verificações antigas e contagens brutas.

Os scripts de operação recebem PAINEL_ID_TOKEN só no ambiente do processo, nunca na linha de argumentos ou em documentos públicos. É um token temporário de sessão, que deve ser introduzido por meio seguro e removido após o uso. Não colocar em chats, ficheiros versionados ou histórico de terminal.

```
node scripts/backup-account.mjs --url http://127.0.0.1:8787 --out /caminho/privado/conta.painel-backup
    .json
node scripts/restore-account.mjs --url http://127.0.0.1:8787 --file /caminho/privado/conta.painel-back
    up.json
```

O backup usa criação exclusiva (não substitui ficheiro existente), permissão 0600 e SHA-256. O restauro começa como prévia: --apply executa, --allow-new-account permite migração intencional para outro UID, --resume repõe apenas IDs em falta e rejeita valores diferentes. Servidores remotos exigem HTTPS e --production. Não executar estes exemplos sem o token da conta e a verificação do destino.

O restauro preserva IDs, custos capturados e stock, mas não reativa login, credenciais, assinatura, jobs ou configuração da taxa padrão da conta. Reconfigurar o padrão no destino; os valores já capturados nas vendas permanecem. Em falha parcial, usar o procedimento de retoma validado em vez de apagar dados para tornar o destino vazio.

Exportação de uma conta não é backup administrativo integral de Firestore/Authentication. Não existe backup automático integral de produção nesta versão. Definir armazenamento privado, retenção e recuperação com o proprietário antes de prometer recuperação a clientes.

21. Histórico consolidado e estado de confirmação

Período	Resultado incorporado nesta edição
Outubro de 2026, fase inicial	Auth/Firestore isolado, Workers, importação Shopify/Meta, quota e recuperação de sessão, catálogo/aliases/stock independente e correções de vendas.
13/10/2026	Concluir processado e pago, gráficos de linhas/acumulado, instalação PWA e relatórios operacionais. A contagem real anual continuou por conferir.
15/10/2026	Preparação comercial: administração/validade, paginação, qualidade, snapshots de mercadoria, deduplicação, privacidade/encerramento, histórico e backup/restauro.
16/10/2026 e revisões seguintes	Produtos na navegação principal; Gastos com registo em diálogo; cartões móveis compactos; Lista, Entregas e Relatórios separados. Protótipos intermédios com meses anteriores/organização embutida não são a UI atual.
05/11/2026	Província em importação/edição/pesquisa/CSV/PDF. Novo app Shopify de uso próprio criado pelo proprietário/outro ambiente, com transição ainda a confirmar.
06/11/2026, código 53fa8d7	Taxa padrão por pedido concluído, exceções, preservação de snapshots, rateio e testes Node/Workerd/navegador. O proprietário confirmou que a opção estava publicada.

O relatório comercial de 15/10 e as notas históricas UI/UX não devem ser lidos como descrições concorrentes da versão atual. Este manual reuniu o conteúdo funcional/técnico e corrigiu descrições substituídas. Logs e testes antigos comprovam intervenções nas suas datas, não a configuração atual de cada plataforma.

Acesso de desenvolvimento ao endereço público chegou a ser recusado pelo proxy do ambiente. Isso não prova bloqueio da loja, do painel ou da conta do proprietário. Publicação deve ser conferida por commit/deployment, painel-build e sessão autorizada; não confundir um teste local ou erro de rede do programador com evidência de produção.

Continuam sem confirmação independente neste documento: totais reais de vendas anuais do proprietário, aprovações de clientes externos, estado final da transição para o novo app Shopify, backups automáticos integrais e execução remota atual do workflow. Não transformar planos em funcionalidades certificadas.

22. Manutenção deste documento e ficheiro para download

Esta edição é autocontida: reúne as decisões vigentes, operação comercial, segurança/privacidade, recuperação, mudanças UI/UX, província e custos de entrega. Documentos antigos foram usados como fontes históricas, sem reproduzir instruções já substituídas. Para trabalhar, usar o código/schema e este manual como referência; validar configurações externas com o proprietário.

A versão Markdown é a fonte editável e o PDF é gerado por scripts/export-docs.py. Requer Python, ReportLab e fontes DejaVu disponíveis no sistema. A geração não depende de contas Firebase/Shopify/Meta.

```
python scripts/export-docs.py docs/MANUAL-TECNICO-E-FUNCIONAL.md
```

Verificar sumário, caracteres, tabelas, páginas partidas e links do ficheiro final antes de publicar. Após alterar regras relevantes, atualizar as secções correspondentes, a data e o commit de referência, e regenerar o PDF. Manter uma cópia datada para identificar o download sem ambiguidade de cache. Nunca inserir dados reais de clientes, tokens ou Secrets em nenhum dos formatos.